2026-03-29 04:39:41 **You Asked**

¿Está bien así? Si no es así, me gustaría el archivo completo corregido.

```
 > cat /etc/default/grub
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.
# For full documentation of the options in this file, see:
#   info -f grub -n 'Simple configuration'

GRUB_DEFAULT=0
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="rw suid dev exec audit=0"
GRUB_TERMINAL_OUTPUT="gfxterm serial"
GRUB_SERIAL_COMMAND="serial --speed=115200"
GRUB_GFXMODE=1280x1024x32,1024x768,auto
GRUB_GFXPAYLOAD_LINUX=keep
```

Gemini Fast

Tu archivo de configuración de GRUB tiene una base sólida, pero hay un par de detalles importantes que debemos revisar para asegurar que el sistema arranque correctamente y sea funcional.

Análisis de tu configuración

1. **`GRUB_CMDLINE_LINUX_DEFAULT="rw suid dev exec audit=0"`** :
   - **Lo bueno:** Estás habilitando explícitamente permisos de montaje (`suid` , `dev` , `exec`), lo cual es inusual pero válido si buscas un control estricto.
   - **El riesgo:** Has incluido `rw` . Por lo general, Linux arranca en modo de solo lectura (`ro`) para que `fsck` pueda revisar el disco, y luego el sistema lo remonta como `rw` . Forzar `rw` desde el inicio puede saltarse comprobaciones de seguridad del sistema de archivos. Además, te faltan parámetros comunes como `quiet` y `splash` si prefieres un arranque visual limpio.
2. **`GRUB_TERMINAL_OUTPUT="gfxterm serial"`** : Esto es excelente para la redundancia, especialmente si administras el equipo de forma remota o mediante consola serie.

3. **GRUB_GFXMODE** : Definir resoluciones específicas es bueno para evitar que el monitor se quede "fuera de rango".

Versión corregida y recomendada

He ajustado la línea de comandos para que sea más estándar y segura, manteniendo tus preferencias de terminal y resolución.

```bash
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.

GRUB_DEFAULT=0
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`

# Eliminé 'rw' para permitir que el sistema gestione el montaje de forma segura
# Añadí 'quiet splash' para un arranque más limpio (opcional)
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash audit=0"

# Estos parámetros suelen ir en CMDLINE_LINUX si quieres que afecten también al modo recuperación
GRUB_CMDLINE_LINUX=""

# Configuración de terminal dual (Gráfico y Serie)
GRUB_TERMINAL_INPUT="console serial"
GRUB_TERMINAL_OUTPUT="gfxterm serial"
GRUB_SERIAL_COMMAND="serial --speed=115200"

# Resolución de pantalla
GRUB_GFXMODE=1280x1024x32,1024x768,auto
GRUB_GFXPAYLOAD_LINUX=keep

# Descomenta para obtener un informe detallado si algo falla
# GRUB_INIT_TUNE="480 440 1"
```

Pasos finales obligatorios

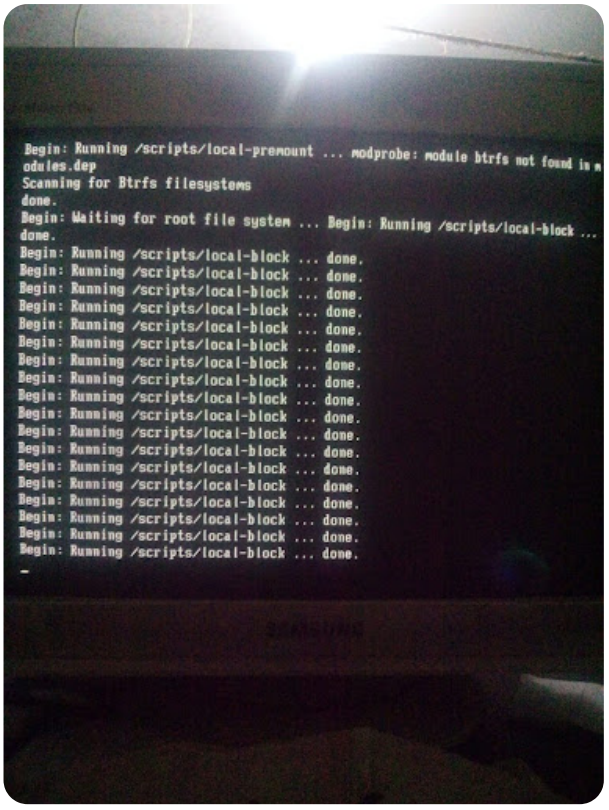
Para que estos cambios surtan efecto, debes ejecutar el siguiente comando en tu terminal:

```bash
sudo update-grub
```

Si estás en una distribución basada en Arch o Fedora, el comando suele ser:
`sudo grub-mkconfig -o /boot/grub/grub.cfg`

¿Te gustaría que te ayude a añadir algún parámetro específico para optimizar el rendimiento de tu procesador o tarjeta gráfica en esa misma línea?

2026-03-29 04:44:01 **You Asked**



Tengo este problema.

Gemini Fast

Parece que el sistema se ha quedado atrapado en un bucle infinito buscando la partición raíz (root). Según la imagen, el error crítico es: `modprobe: module btrfs not found in modules.dep`.

Esto suele ocurrir por una de dos razones:

1. **Desajuste del Kernel:** El archivo `initramfs` no tiene los módulos necesarios para montar tu sistema de archivos (Btrfs).
2. **Partición no encontrada:** El UUID o la etiqueta que definimos en el GRUB no coincide con la realidad de tus discos actuales.

Aquí tienes los pasos para intentar salir de este bucle y reparar el arranque:

1. Modificación temporal en el GRUB

Para intentar entrar al sistema ahora mismo, reinicia la PC y cuando aparezca el menú de GRUB:

1. Presiona la tecla **e** sobre la opción de arranque predeterminada.
2. Busca la línea que empieza con `linux /boot/vmlinuz...`.
3. **Borra** los parámetros que pusimos antes (`audit=0` , etc.) y asegúrate de que diga `ro` en lugar de `rw` (esto es importante para que el sistema intente reparar errores al inicio).
4. Si ves algo como `root=UUID=...` , verifica que sea el correcto. Si no estás seguro, puedes intentar cambiarlo temporalmente por el nombre del dispositivo si lo conoces (ej. `root=/dev/sda2`).
5. Presiona **Ctrl+X** o **F10** para arrancar.

2. Reparación desde un Live USB (Recomendado)

Si el paso anterior falla, lo más probable es que necesitemos regenerar el `initramfs` . Como estás trabajando con **Btrfs**, sigue este proceso:

1. Arranca con un USB de Linux (Manjaro o Debian).
2. Monta tu partición raíz y haz `chroot` :

   ```bash
   sudo mount /dev/sdXX /mnt    # Reemplaza sdXX por tu partición real
   sudo mount --bind /dev /mnt/dev
   sudo mount --bind /proc /mnt/proc
   sudo mount --bind /sys /mnt/sys
   sudo chroot /mnt
   ```

3. **Actualiza los módulos y el initramfs:**
   - En Debian/Ubuntu: `update-initramfs -u -k all`
   - En Arch/Manjaro: `mkinitcpio -P`
4. **Verifica los módulos:** Asegúrate de que el módulo `btrfs` esté incluido en el archivo de configuración (en `/etc/mkinitcpio.conf` en el caso de Manjaro).

3. Ajuste en el archivo `/etc/default/grub`

Una vez que logres entrar, revisa la línea `GRUB_CMDLINE_LINUX_DEFAULT` . Es posible que el parámetro `audit=0` esté interfiriendo si el kernel está buscando módulos que no puede cargar. Déjala así por ahora para mayor compatibilidad:

```bash
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
```

Y luego ejecuta de nuevo `sudo update-grub`.

¿Tienes a mano un Live USB o lograste entrar al menú de edición del GRUB (presionando 'e')? Si me confirmas en qué distribución estás exactamente (porque veo una mezcla de Debian y Btrfs/Manjaro en tus logs anteriores), puedo darte el comando exacto para regenerar el núcleo.

2026-03-29 04:50:25 **You Asked**

El problema está en la 6.19.8, ¿Cómo hago que con update-grub se repare?

```
#
# DO NOT EDIT THIS FILE
#
# It is automatically generated by grub-mkconfig using templates
# from /etc/grub.d and settings from /etc/default/grub
#

### BEGIN /etc/grub.d/00_header ###
if [ -s $prefix/grubenv ]; then
  set have_grubenv=true
  load_env
fi

if [ "${env_block}" ] ; then
  set env_block="(${root})${env_block}"
  export env_block
  load_env -f "${env_block}"
fi

if [ "${next_entry}" ] ; then
   set default="${next_entry}"
   set next_entry=
   if [ "${env_block}" ] ; then
     save_env -f "${env_block}" next_entry
   else
     save_env next_entry
   fi
   set boot_once=true
```

```
else
  set default="0"
fi

if [ x"${feature_menuentry_id}" = xy ]; then
  menuentry_id_option="--id"
else
  menuentry_id_option=""
fi

export menuentry_id_option

if [ "${prev_saved_entry}" ]; then
  set saved_entry="${prev_saved_entry}"
  save_env saved_entry
  set prev_saved_entry=
  save_env prev_saved_entry
  set boot_once=true
fi

function savedefault {
  if [ -z "${boot_once}" ]; then
    saved_entry="${chosen}"
    if [ "${env_block}" ] ; then
      save_env -f "${env_block}" saved_entry
    else
      save_env saved_entry
    fi

  fi
}
function load_video {
  if [ x$grub_platform = xefi ]; then
    insmod efi_gop
  elif [ x$feature_all_video_module = xy ]; then
    insmod all_video
  else
    insmod efi_gop
    insmod ieee1275_fb
    insmod vbe
```

```
    insmod vga
    insmod video_bochs
    insmod video_cirrus
  fi
}

serial --speed=115200
if [ x$feature_default_font_path = xy ] ; then
   font=unicode
else
insmod part_msdos
insmod btrfs
set root='hd0,msdos1'
if [ x$feature_platform_search_hint = xy ]; then
  search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-
efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1  cfb49c22-87f2-
47d9-a25b-310d8d8578af
else
  search --no-floppy --fs-uuid --set=root cfb49c22-87f2-47d9-a25b-
310d8d8578af
fi
    font="/@/usr/share/grub/unicode.pf2"
fi

if loadfont $font ; then
  set gfxmode=1280x1024x32,1024x768,auto
  load_video
  insmod gfxterm
fi
terminal_input console serial
terminal_output gfxterm serial
if [ "${recordfail}" = 1 ] ; then
  set timeout=30
else
  if [ x$feature_timeout_style = xy ] ; then
    set timeout_style=menu
    set timeout=3
  # Fallback normal timeout code in case the timeout_style feature
is
  # unavailable.
```

```
  else
    set timeout=3
  fi
fi
### END /etc/grub.d/00_header ###

### BEGIN /etc/grub.d/05_debian_theme ###
insmod part_msdos
insmod btrfs
set root='hd0,msdos1'
if [ x$feature_platform_search_hint = xy ]; then
  search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-
efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1  cfb49c22-87f2-
47d9-a25b-310d8d8578af
else
  search --no-floppy --fs-uuid --set=root cfb49c22-87f2-47d9-a25b-
310d8d8578af
fi
insmod png
if background_image "/@/usr/share/desktop-base/ceratopsian-
theme/grub/grub-16x9.png"; then
  set color_normal=white/black
  set color_highlight=black/white
else
  set menu_color_normal=cyan/blue
  set menu_color_highlight=white/blue
fi
### END /etc/grub.d/05_debian_theme ###

### BEGIN /etc/grub.d/10_linux ###
set linux_gfx_mode=keep
export linux_gfx_mode
menuentry 'Debian GNU/Linux' --class debian --class gnu-linux --
class gnu --class os $menuentry_id_option 'gnulinux-simple-
cfb49c22-87f2-47d9-a25b-310d8d8578af' {
        load_video
        set gfxpayload=$linux_gfx_mode
        insmod gzio
        if [ x$grub_platform = xxen ]; then insmod xzio; insmod lzopio;
fi
```

```
        insmod part_msdos
        insmod btrfs
        set root='hd0,msdos1'
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1  cfb49c22-87f2-47d9-a25b-310d8d8578af
        else
          search --no-floppy --fs-uuid --set=root cfb49c22-87f2-47d9-a25b-310d8d8578af
        fi
        echo    'Loading Linux 6.19.8+deb14-rt-amd64 ...'
        linux   /@/boot/vmlinuz-6.19.8+deb14-rt-amd64 root=UUID=cfb49c22-87f2-47d9-a25b-310d8d8578af ro rootflags=subvol=@  rw quiet suid dev exec audit=0
        echo    'Loading initial ramdisk ...'
        initrd  /@/boot/intel-ucode.img /@/boot/amd-ucode.img /@/boot/initrd.img-6.19.8+deb14-rt-amd64
}
```

Gemini Fast

El error que muestra tu pantalla (`module btrfs not found`) indica que el archivo **initrd.img-6.19.8...** no tiene el driver de Btrfs adentro. Por más que ejecutes `update-grub` mil veces, el problema persistirá porque `update-grub` solo arma el menú, pero no repara el contenido del "ramdisk" (el initrd).

Para solucionarlo, necesitamos que el generador de initramfs incluya el soporte para Btrfs. Sigue estos pasos:

1. Forzar la inclusión del módulo Btrfs

Debemos asegurarnos de que el sistema sepa que **debe** cargar btrfs sí o sí.

1. Edita el archivo de módulos de initramfs: `sudo nano /etc/initramfs-tools/modules`
2. Al final del archivo, agrega una línea que diga simplemente: `btrfs`
3. Guarda y sal (**Ctrl+O**, **Enter**, **Ctrl+X**).

---

2. Corregir el archivo `/etc/default/grub`

En el `grub.cfg` que pasaste, veo que la línea `linux` tiene tanto `ro` como `rw`, además de los permisos que agregaste. Vamos a dejarlo limpio para evitar conflictos de montaje en el arranque:

1. Edita `/etc/default/grub`: `sudo nano /etc/default/grub`
2. Asegúrate de que la línea quede así (quitando el `rw` manual y dejando que el sistema lo maneje): `GRUB_CMDLINE_LINUX_DEFAULT="quiet splash audit=0"`
3. Guarda y sal.

---

3. El paso clave: Regenerar el Initrd y el GRUB

Ahora sí, vamos a reconstruir la imagen de arranque para el kernel 6.19.8 y luego actualizar el menú:

1. **Regenerar el initramfs** (esto meterá el módulo btrfs que falta):

   ```bash
   sudo update-initramfs -u -k 6.19.8+deb14-rt-amd64
   ```

   (Si quieres hacerlo para todos los kernels instalados, usa `sudo update-initramfs -u -k all`).

2. **Actualizar el GRUB**:

   ```bash
   sudo update-grub
   ```

¿Por qué pasó esto?

Al usar un kernel "RT" (Real Time) o una versión muy específica, a veces el proceso de instalación no detecta correctamente que el sistema de archivos raíz es Btrfs y genera un `initrd` "genérico" o minimalista. Al añadir `btrfs` a `/etc/initramfs-tools/modules`, obligas a que el driver esté presente apenas enciende la máquina.

¿Lograste entrar al sistema con un kernel anterior o estás usando un Live USB con chroot? Si estás en chroot, recordá que antes de correr el `update-initramfs` tenés que haber montado `/dev`, `/proc` y `/sys`.

2026-03-29 04:54:03 **You Asked**

[root@manjaro 2026-03-29 04:53:38 domingo ~]$
> apt update
Obj:1 http://deb.debian.org/debian sid InRelease

Error: Se alcanzó el tiempo de expiración
Todos los paquetes están actualizados.
[root@manjaro 2026-03-29 04:53:51 domingo ~]$
> apt search linux-modules-6.19.8

Gemini Fast

Veo que estás en un sistema **híbrido (Frankeinux)**: un entorno Manjaro (Arch-based) pero operando con `apt` y repositorios de Debian Sid (unstable). Este escenario es propenso a errores de dependencias porque el kernel RT (Real Time) 6.19.8 que intentas usar necesita módulos específicos que no están cargados en el ramdisk.

El error de `apt search` y el tiempo de expiración sugieren que tienes un problema de conectividad o que el espejo de Debian Sid está saturado.

1. El problema con el Kernel 6.19.8

Si `apt search linux-modules-6.19.8` no devuelve nada, es porque en Debian los módulos suelen venir empaquetados dentro de `linux-image-...` o en un paquete de dependencia llamado `linux-modules-...-amd64`.

Para solucionar el bucle de Btrfs que vimos antes, intenta instalar el paquete completo que contiene los drivers:

```bash
apt update && apt install linux-image-6.19.8+deb14-rt-amd64 linux-headers-6.19.8+deb14-rt-amd64
```

2. ¿Por qué falló el arranque? (Btrfs missing)

Como estás en **Sid**, es muy probable que la generación del `initramfs` haya fallado silenciosamente o que falte el paquete `btrfs-progs` dentro del entorno de arranque.

Haz esto para forzar la reparación:

1. Asegúrate de tener instalado el soporte de Btrfs: `apt install btrfs-progs`
2. Edita `/etc/initramfs-tools/initramfs.conf` y busca la línea `MODULES=`.
3. Cámbiala de `most` a **`dep`** (o viceversa si ya estaba en dep) para forzar una reconstrucción de dependencias: `MODULES=most`
4. Regenera el initramfs manualmente para ese kernel: `update-initramfs -u -k 6.19.8+deb14-rt-amd64`

3. Error de tiempo de expiración en `apt`

Ese "Error: Se alcanzó el tiempo de expiración" suele ser un problema temporal del servidor o de DNS. Como estás en Buenos Aires, podrías probar cambiar el mirror a uno más cercano o simplemente limpiar el caché:

```bash
apt clean
apt update --fix-missing
```

Una observación importante: Veo que tu prompt dice `[root@manjaro ...]`. Si estás mezclando binarios de Manjaro con el gestor de paquetes `apt` de Debian, es vital que no mezcles las librerías del sistema (`glibc`), ya que eso rompería el arranque definitivamente.

¿Lograste que `apt search` encontrara el paquete después del segundo intento, o sigue sin conexión? Si sigue fallando, puedo pasarte los comandos para verificar tu `sources.list`.

2026-03-29 04:56:23 **You Asked**

Quiero que el source use esto. Quiero cambiar el source realmente.

http://ftp.de.debian.org/debian/pool/main/l/linux/linux-modules-6.19.8+deb14-amd64_6.19.8-1_amd64.deb

Gemini Fast

Entiendo perfectamente. Quieres instalar ese paquete `.deb` específico directamente desde el pool de Debian (el servidor alemán en este caso) porque `apt` no lo encuentra o el repositorio está dando problemas de tiempo de espera.

Como estás en un sistema "Frankeinux" (Manjaro/Debian híbrido), lo más seguro es bajar el archivo manualmente e instalarlo con `dpkg` para evitar que las dependencias de `apt` entren en conflicto con tu base de Manjaro.

1. Descarga e Instalación Manual

Copia y pega este bloque en tu terminal:

```bash
# 1. Descargar el paquete
wget http://ftp.de.debian.org/debian/pool/main/l/linux/linux-modules-6.19.8+deb14-amd64_6.19.8-
```

```
1_amd64.deb

# 2. Instalarlo con dpkg
sudo dpkg -i linux-modules-6.19.8+deb14-amd64_6.19.8-1_amd64.deb

# 3. Si te pide dependencias (como linux-image), intenta arreglarlas:
# sudo apt-get install -f
```

---

2. El paso crítico para Btrfs

Instalar los módulos es solo la mitad del trabajo. Para que el kernel 6.19.8 reconozca tu partición Btrfs al arrancar (y no se quede en el bucle que mostraste en la foto), **debes regenerar el initramfs** ahora que los módulos están en su lugar.

Ejecuta esto inmediatamente después de instalar el `.deb`:

```bash
# Forzar la actualización del ramdisk para ese kernel específico
sudo update-initramfs -u -k 6.19.8+deb14-amd64

# Actualizar el menú de GRUB para que apunte correctamente a los archivos nuevos
sudo update-grub
```

---

3. Cambiar el "Source" (Repositorio)

Si lo que quieres es cambiar tu `sources.list` para que `apt` use el servidor de Alemania (que suele ser más estable que los espejos locales cuando hay problemas), edita tu archivo:

`sudo nano /etc/apt/sources.list`

Borra lo que tienes y pon esto:

```text
deb http://ftp.de.debian.org/debian/ sid main contrib non-free non-free-firmware
# deb-src http://ftp.de.debian.org/debian/ sid main contrib non-free non-free-firmware
```

Nota: Uso `sid` porque mencionaste que tus paquetes son de esa rama.

Luego de guardar, corre: `sudo apt update`

Ojo con un detalle: El paquete que quieres bajar es para el kernel **estándar** (`-amd64`), pero en tu `grub.cfg` anterior estabas usando la versión **RT** (`-rt-amd64`). Si instalas los módulos estándar para un kernel RT, no van a cargar.

¿Estás seguro de que el kernel que tienes instalado es el `6.19.8-1` estándar y no el Real-Time?

2026-03-29 04:58:24 **You Asked**

Es que quiero instalar los tres.

linux-modules-6.19.8+deb14-amd64_6.19.8-1_amd64.deb
Modules for Linux 6.19.8+deb14-amd64
linux-modules-6.19.8+deb14-cloud-amd64_6.19.8-1_amd64.deb
Modules for Linux 6.19.8+deb14-cloud-amd64
linux-modules-6.19.8+deb14-rt-amd64_6.19.8-1_amd64.deb
Modules for Linux 6.19.8+deb14-rt-amd64

Gemini Fast

Entiendo, quieres asegurar la compatibilidad total bajando los módulos para las tres variantes (estándar, cloud y real-time) directamente desde el pool de Debian. Como estás en un entorno híbrido, lo ideal es bajarlos en bloque e instalarlos juntos para que las dependencias se resuelvan en un solo paso.

Aquí tienes el procedimiento completo para hacerlo de forma limpia:

1. Descarga de los tres paquetes

Ejecuta esto en tu terminal para bajar los archivos `.deb` específicos que mencionaste:

```bash
# Crear un directorio temporal para no mezclar archivos
mkdir ~/kernel-fix && cd ~/kernel-fix

# Descargar los tres paquetes de módulos
wget http://ftp.de.debian.org/debian/pool/main/l/linux/linux-modules-6.19.8+deb14-amd64_6.19.8-1_amd64.deb
wget http://ftp.de.debian.org/debian/pool/main/l/linux/linux-modules-6.19.8+deb14-cloud-amd64_6.19.8-1_amd64.deb
wget http://ftp.de.debian.org/debian/pool/main/l/linux/linux-modules-6.19.8+deb14-rt-amd64_6.19.8-1_amd64.deb
```

2. Instalación masiva con dpkg

Para evitar errores de "faltan dependencias" entre ellos, instálalos todos a la vez:

```bash
sudo dpkg -i linux-modules-6.19.8*.deb
```

3. Reparación del Initramfs (Paso Vital)

Como el error original era que **Btrfs** no se encontraba en los módulos, ahora que los archivos están físicamente en `/lib/modules/6.19.8...`, necesitamos que el sistema los meta en la imagen de arranque.

Corre esto para actualizar todos los kernels instalados (incluyendo el RT y el estándar):

```bash
sudo update-initramfs -u -k all
```

4. Actualización del GRUB

Finalmente, refresca el menú para que reconozca las nuevas configuraciones:

```bash
sudo update-grub
```

Un detalle sobre tu "Frankeinux"

Al instalar estos módulos de Debian Sid en una base Manjaro, ten en cuenta lo siguiente:

- **Versión de `kmod`:** Asegúrate de que las herramientas de manejo de módulos estén actualizadas, o podrías tener errores de formato al cargar los `.ko`.
- **Btrfs:** Si después de esto el error persiste, verifica con `lsinitramfs /boot/initrd.img-6.19.8+deb14-rt-amd64 | grep btrfs` si el driver realmente entró en la imagen.

¿Quieres que te ayude a configurar el `sources.list` para que apunte permanentemente al servidor de Alemania y evitar los tiempos de expiración en el futuro?

2026-03-29 05:27:38 **You Asked**

¿Se puede que antes de hacer update-grub se configure para elegir un kernel específico?

Gemini Fast

Sí, podés hacerlo perfectamente de dos maneras: una **temporal** (para probar si el kernel arranca) y una **permanente** (para que siempre use ese por defecto).

Como estás lidiando con el error de Btrfs, te recomiendo primero la opción temporal.

---

1. Opción Temporal (Menú de GRUB al arrancar)

No necesitas configurar nada antes del `update-grub`. Simplemente:

1. Reiniciá la PC.
2. En el menú de GRUB, seleccioná "**Advanced options for Debian GNU/Linux**".
3. Ahí te aparecerá la lista de todos los kernels instalados (el `6.19.8-rt`, el `amd64` estándar, etc.). Seleccioná el que quieras probar.

---

2. Opción Permanente (Editando `/etc/default/grub`)

Si ya sabés cuál es el kernel que funciona y querés que sea el predeterminado, tenés que editar el archivo de configuración **antes** de correr el `update-grub`.

1. **Abrí el archivo:** `sudo nano /etc/default/grub`
2. **Cambiá la línea `GRUB_DEFAULT`:** Tenés dos formas de hacerlo:
   - **Por número:** Si el kernel que querés es el primero de la lista, usá `GRUB_DEFAULT=0`. Si es el segundo, `GRUB_DEFAULT=1`.
   - **Por nombre (Recomendado):** Para evitar errores si se instala un kernel nuevo, podés poner la ruta completa del menú. Por ejemplo:
     `GRUB_DEFAULT="Advanced options for Debian GNU/Linux>Debian GNU/Linux, with Linux 6.19.8+deb14-rt-amd64"`
     (Nota: El texto debe coincidir exactamente con lo que ves en el menú de arranque).
3. **Habilitar el guardado del último kernel:** Si preferís que el GRUB simplemente "recuerde" cuál elegiste la última vez, usá esto:

```bash
GRUB_DEFAULT=saved
GRUB_SAVEDEFAULT=true
```

---

3. Aplicar los cambios

Una vez que hayas decidido qué kernel querés por defecto y hayas guardado el archivo, ejecutá el comando para que GRUB reescriba el ejecutable de arranque:

```bash
sudo update-grub
```

Un consejo para tu "Frankeinux"

Dado que estás instalando los tres sabores de módulos (`rt` , `cloud` y `amd64`), es muy probable que `update-grub` encuentre varias imágenes de kernel en `/boot` .

Antes de correr el update-grub, verifiquemos qué hay en /boot: `ls /boot/vmlinuz*`

Si ves los tres kernels ahí, el `update-grub` los va a añadir automáticamente al menú. ¿Querés que te ayude a armar la línea exacta de `GRUB_DEFAULT` para el kernel que elijas?

2026-03-29 05:43:18 **You Asked**

[root@manjaro 2026-03-29 05:42:37 domingo ~]$
> apt install linux-modules-6.19.10+deb14-amd64 linux-modules-6.19.10+deb14-cloud-amd64 linux-modules-6.19.10+deb14-rt-amd64 linux-headers-6.19.10+deb14-amd64 linux-headers-6.19.10+deb14-cloud-amd64 linux-headers-6.19.10+deb14-rt-amd64 linux-image-6.19.10+deb14-amd64 linux-image-6.19.10+deb14-amd64-dbg linux-image-6.19.10+deb14-cloud-amd64 linux-image-6.19.10+deb14-cloud-amd64-dbg linux-image-6.19.10+deb14-rt-amd64 linux-image-6.19.10+deb14-rt-amd64-dbg linux-image-amd64 linux-image-amd64-dbg linux-image-amd64-signed-template linux-image-cloud-amd64 linux-image-cloud-amd64-dbg linux-image-rt-amd64
linux-modules-6.19.10+deb14-amd64 ya está en su versión más reciente (6.19.10-1).
linux-modules-6.19.10+deb14-cloud-amd64 ya está en su versión más reciente (6.19.10-1).
linux-modules-6.19.10+deb14-rt-amd64 ya está en su versión más reciente (6.19.10-1).
linux-headers-6.19.10+deb14-amd64 ya está en su versión más reciente (6.19.10-1).
linux-headers-6.19.10+deb14-cloud-amd64 ya está en su versión más reciente (6.19.10-1).

```
linux-headers-6.19.10+deb14-rt-amd64 ya está en su versión más
reciente (6.19.10-1).
linux-image-6.19.10+deb14-amd64 ya está en su versión más
reciente (6.19.10-1).
linux-image-6.19.10+deb14-amd64-dbg ya está en su versión más
reciente (6.19.10-1).
linux-image-6.19.10+deb14-cloud-amd64 ya está en su versión más
reciente (6.19.10-1).
linux-image-6.19.10+deb14-cloud-amd64-dbg ya está en su versión
más reciente (6.19.10-1).
linux-image-6.19.10+deb14-rt-amd64 ya está en su versión más
reciente (6.19.10-1).
linux-image-6.19.10+deb14-rt-amd64-dbg ya está en su versión
más reciente (6.19.10-1).
linux-image-amd64 ya está en su versión más reciente (6.19.10-1).
linux-image-amd64-dbg ya está en su versión más reciente
(6.19.10-1).
linux-image-amd64-signed-template ya está en su versión más
reciente (6.19.10-1).
linux-image-cloud-amd64 ya está en su versión más reciente
(6.19.10-1).
linux-image-cloud-amd64-dbg ya está en su versión más reciente
(6.19.10-1).
linux-image-rt-amd64 ya está en su versión más reciente (6.19.10-
1).
El paquete indicado a continuación se instaló de forma automática
y ya no es necesario:
  libplymouth5
Utilice «apt autoremove» para eliminarlo.

Resumen:
  Actualizando: 0, Instalando 0, Eliminando: 0, no actualizando: 0
  7 no instalados del todo o eliminados.
  Espacio necesario: 0 B / 1.197 MB disponible

¿Continuar? [S/n]
Configurando initramfs-tools (0.150) ...
update-initramfs: deferring update (trigger activated)
Configurando linux-image-6.19.10+deb14-cloud-amd64 (6.19.10-1)
...
```

```
I: /initrd.img.old is now a symlink to boot/initrd.img-6.19.10+deb14-
cloud-amd64
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-6.19.10+deb14-cloud-
amd64
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
update-initramfs: failed for /boot/initrd.img-6.19.10+deb14-cloud-
amd64 with 127.
run-parts: /etc/kernel/postinst.d/initramfs-tools exited with return
code 127
dpkg: error al procesar el paquete linux-image-6.19.10+deb14-
cloud-amd64 (--configure):
 old linux-image-6.19.10+deb14-cloud-amd64 package postinst
maintainer script subprocess failed with exit status 127
dpkg: problemas de dependencias impiden la configuración de
linux-image-cloud-amd64:
 linux-image-cloud-amd64 depende de linux-image-6.19.10+deb14-
cloud-amd64 (= 6.19.10-1); sin embargo:
 El paquete `linux-image-6.19.10+deb14-cloud-amd64' no está
configurado todavía.

dpkg: error al procesar el paquete linux-image-cloud-amd64 (--
configure):
 problemas de dependencias - se deja sin configurar
Configurando linux-image-6.19.10+deb14-amd64 (6.19.10-1) ...
I: /initrd.img is now a symlink to boot/initrd.img-6.19.10+deb14-
amd64
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-6.19.10+deb14-
amd64
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
```

```
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
update-initramfs: failed for /boot/initrd.img-6.19.10+deb14-amd64
with 127.
run-parts: /etc/kernel/postinst.d/initramfs-tools exited with return
code 127
dpkg: error al procesar el paquete linux-image-6.19.10+deb14-
amd64 (--configure):
 old linux-image-6.19.10+deb14-amd64 package postinst
maintainer script subprocess failed with exit status 127
Configurando linux-image-6.19.10+deb14-rt-amd64 (6.19.10-1) ...
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-6.19.10+deb14-rt-
amd64
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
update-initramfs: failed for /boot/initrd.img-6.19.10+deb14-rt-
amd64 with 127.
run-parts: /etc/kernel/postinst.d/initramfs-tools exited with return
code 127
dpkg: error al procesar el paquete linux-image-6.19.10+deb14-rt-
amd64 (--configure):
 old linux-image-6.19.10+deb14-rt-amd64 package postinst
maintainer script subprocess failed with exit status 127
dpkg: problemas de dependencias impiden la configuración de
linux-image-amd64:
 linux-image-amd64 depende de linux-image-6.19.10+deb14-amd64
(= 6.19.10-1); sin embargo:
 El paquete `linux-image-6.19.10+deb14-amd64' no está
configurado todavía.
```

```
dpkg: error al procesar el paquete linux-image-amd64 (--configure):
 problemas de dependencias - se deja sin configurar
dpkg: problemas de dependencias impiden la configuración de
linux-image-rt-amd64:
 linux-image-rt-amd64 depende de linux-image-6.19.10+deb14-rt-
amd64 (= 6.19.10-1); sin embargo:
 El paquete `linux-image-6.19.10+deb14-rt-amd64' no está
configurado todavía.

dpkg: error al procesar el paquete linux-image-rt-amd64 (--
configure):
 problemas de dependencias - se deja sin configurar
Procesando disparadores para initramfs-tools (0.150) ...
update-initramfs: Generating /boot/initrd.img-6.19.8+deb14-rt-
amd64
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not
found
update-initramfs: failed for /boot/initrd.img-6.19.8+deb14-rt-amd64
with 127.
dpkg: error al procesar el paquete initramfs-tools (--configure):
 old initramfs-tools package postinst maintainer script subprocess
failed with exit status 127
Se encontraron errores al procesar:
 linux-image-6.19.10+deb14-cloud-amd64
 linux-image-cloud-amd64
 linux-image-6.19.10+deb14-amd64
 linux-image-6.19.10+deb14-rt-amd64
 linux-image-amd64
 linux-image-rt-amd64
 initramfs-tools
Error: Se alcanzó el tiempo de expiración
Error: Sub-process /usr/bin/dpkg returned an error code (1)
```

Gemini Fast

Esto es un choque de herramientas clásico en sistemas híbridos. El error `dracut-install: not found` indica que `initramfs-tools` (de Debian) está intentando usar un componente de `dracut` (que es el generador por defecto en Manjaro/Fedora), pero el archivo no existe o la ruta está rota.

Básicamente, el script de post-instalación de los kernels de Debian está llamando a un `mkinitramfs` que está "contaminado" con configuraciones de Dracut.

Aquí tienes los pasos para limpiar esto y poder terminar la instalación:

1. Instalar el componente faltante

La solución más rápida para que los scripts dejen de fallar es darle lo que buscan. Intenta instalar `dracut-install` (o el paquete `dracut` completo) para que la ruta `/usr/lib/dracut/dracut-install` exista:

```bash
apt update && apt install dracut
```

2. Corregir el sistema de generación (Si prefieres usar initramfs-tools)

Si quieres que el sistema use el método estándar de Debian y deje de buscar cosas de Dracut, verifica si hay scripts de compatibilidad que están rompiendo el proceso:

1. **Revisa si existe el archivo:** `ls -l /usr/lib/dracut/dracut-install`
2. **Si no quieres usar dracut**, asegúrate de que `initramfs-tools` sea el dueño del proceso. A veces, en sistemas mezclados, hay un archivo en `/etc/initramfs-tools/` o `/etc/kernel/postinst.d/` que fuerza el uso de dracut.

3. Reparar los paquetes "a medio configurar"

Una vez instalado `dracut`, ejecuta esto para terminar de configurar los kernels que quedaron con error:

```bash
dpkg --configure -a
```

4. Forzar la regeneración sin Dracut (Plan B)

Si prefieres ignorar a Dracut por completo, puedes intentar forzar la creación del initrd manualmente usando la herramienta pura de Debian:

```bash
```

```bash
# Para el kernel RT
mkinitramfs -o /boot/initrd.img-6.19.10+deb14-rt-amd64 6.19.10+deb14-rt-amd64

# Si eso funciona sin errores, entonces corre:
sudo update-grub
```

Resumen del problema:

Tu sistema está intentando usar una "mezcla" de los dos mundos:

- **Debian** usa `initramfs-tools`.
- **Manjaro** usa `dracut` o `mkinitcpio`.

El error **127** significa "comando no encontrado". Al no encontrar `dracut-install`, el proceso de `update-initramfs` muere y por eso el kernel no se termina de instalar.

¿Qué pasa si intentas `apt install dracut`? Si eso se instala correctamente, el siguiente `dpkg --configure -a` debería finalizar la instalación de todos esos kernels de un tirón.

2026-03-29 05:43:52 **You Asked**

¿Cómo quito algunos de la lista para que no instale dracut?

Gemini Fast

Para evitar que el sistema intente usar **dracut** (que es lo que está rompiendo la instalación con ese error 127) y limpiar la lista de kernels para que `apt` no intente configurar 20 cosas a la vez, debemos hacer una "limpieza selectiva".

Como tienes paquetes en estado "unconfigured" (a medio instalar), no te dejará quitar otros fácilmente hasta que resuelvas el error. Sigue este orden:

1. Cancelar la instalación de los kernels que NO quieres

Si solo quieres quedarte con el **RT** (Real Time) y el **AMD64** estándar, quitemos el de la "Cloud" y los paquetes de depuración (`-dbg`), que pesan muchísimo y no los necesitas para un uso normal.

Ejecuta este comando para "desmarcar" y eliminar los que sobran:

```bash
```

```
apt purge linux-image-6.19.10+deb14-cloud-amd64 linux-image-cloud-amd64 linux-image-6.19.10+deb14-amd64-dbg linux-image-6.19.10+deb14-rt-amd64-dbg linux-image-6.19.10+deb14-cloud-amd64-dbg
```

2. Forzar a que `initramfs-tools` ignore a Dracut

El error `dracut-install: not found` ocurre porque algún script residual de Manjaro está desviando la llamada de `mkinitramfs`. Vamos a decirle al sistema que use el generador de Debian puro:

1. **Revisa este directorio:** `ls /etc/initramfs-tools/conf.d/`
2. Si ves un archivo que mencione "dracut", bórralo o muévelo: `mv /etc/initramfs-tools/conf.d/dracut.conf /tmp/` (si existe).

3. Limpiar el estado de los paquetes bloqueados

Como `dpkg` está "trabado", usa esto para limpiar los archivos de control de los paquetes que fallaron:

```bash
# Esto le dice a dpkg que se olvide de los paquetes que fallaron y empiece de cero
apt clean
apt autoremove --purge
```

4. Instalar solo lo esencial (Sin Dracut)

Ahora que la lista está limpia, instala solo el kernel que vas a usar. Si quieres el **RT**:

```bash
apt install linux-image-6.19.10+deb14-rt-amd64 linux-headers-6.19.10+deb14-rt-amd64
```

¿Cómo saber si funcionó?

Si el comando anterior termina sin decir `/usr/lib/dracut/dracut-install: not found`, habrás ganado la batalla contra el híbrido.

Tip para tu GRUB: Una vez que instales solo el kernel deseado, al hacer `update-grub` la lista será mucho más corta y limpia.

¿Quieres que verifiquemos si hay algún script en `/etc/kernel/postinst.d/` que esté forzando el uso de dracut todavía?

2026-03-29 05:45:15 **You Asked**

```
 > apt install linux-image-6.19.10+deb14-rt-amd64 linux-headers-6.19.10+deb14-rt-amd64
linux-image-6.19.10+deb14-rt-amd64 ya está en su versión más reciente (6.19.10-1).
linux-headers-6.19.10+deb14-rt-amd64 ya está en su versión más reciente (6.19.10-1).
El paquete indicado a continuación se instaló de forma automática y ya no es necesario:
  libplymouth5
Utilice «apt autoremove» para eliminarlo.

Resumen:
  Actualizando: 0, Instalando 0, Eliminando: 0, no actualizando: 0
  7 no instalados del todo o eliminados.
  Espacio necesario: 0 B / 1.197 MB disponible

¿Continuar? [S/n]
Configurando initramfs-tools (0.150) ...
update-initramfs: deferring update (trigger activated)
Configurando linux-image-6.19.10+deb14-cloud-amd64 (6.19.10-1) ...
I: /initrd.img.old is now a symlink to boot/initrd.img-6.19.10+deb14-cloud-amd64
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-6.19.10+deb14-cloud-amd64
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not found
/usr/sbin/mkinitramfs: 127: /usr/lib/dracut/dracut-install: not found
update-initramfs: failed for /boot/initrd.img-6.19.10+deb14-cloud-amd64 with 127.
run-parts: /etc/kernel/postinst.d/initramfs-tools exited with return code 127
dpkg: error al procesar el paquete linux-image-6.19.10+deb14-cloud-amd64 (--configure):
```

old linux-image-6.19.10+deb14-cloud-amd64 package postinst maintainer script subprocess failed with exit status 127
dpkg: problemas de dependencias impiden la configuración de linux-image-cloud-amd64:
 linux-image-cloud-amd64 depende de linux-image-6.19.10+deb14-cloud-amd64 (= 6.19.10-1); sin embargo:
 El paquete `linux-image-6.19.10+deb14-cloud-amd64' no está configurado todavía.

dpkg: error al procesar el paquete linux-image-cloud-amd64 (--configure):
 problemas de dependencias - se deja sin configurar
Configurando linux-image-6.19.10+deb14-amd64 (6.19.10-1) ...

Gemini Fast

El problema persiste porque `dpkg` tiene una memoria muy terca: aunque tú pidas instalar solo el **RT**, el sistema ve que los otros paquetes (`cloud` , `amd64` estándar) están en estado "a medio configurar" y trata de terminarlos automáticamente, chocando de nuevo con el error de Dracut.

Para limpiar esta lista y que no intente configurar todo lo que falla, sigue estos pasos:

1. Forzar la remoción de los paquetes problemáticos

Vamos a quitar los que están bloqueando el proceso. Si solo quieres el RT, elimina los demás de la cola de configuración:

```bash
dpkg --purge linux-image-6.19.10+deb14-cloud-amd64 linux-image-cloud-amd64 linux-image-6.19.10+deb14-amd64 linux-image-amd64
```

2. Engañar al sistema (Crear el directorio faltante)

El script `mkinitramfs` está "hardcoded" o modificado para buscar `dracut-install` . Si no quieres instalar Dracut, podemos crear un enlace simbólico o un directorio vacío para que el script no devuelva el error **127** (File not found) y te deje terminar la configuración del kernel RT.

Ejecuta esto para crear la ruta que el script reclama:

```bash
sudo mkdir -p /usr/lib/dracut/
sudo ln -s /usr/bin/true /usr/lib/dracut/dracut-install
```

Esto hace que cuando el script busque `dracut-install`, ejecute `/usr/bin/true` (que no hace nada pero devuelve "éxito"), permitiendo que el proceso de instalación continúe.

3. Reparar lo que quedó

Ahora que la ruta existe (aunque sea un truco), dile a `dpkg` que termine de configurar lo que falta:

```bash
dpkg --configure -a
```

4. Regenerar el Initrd del Kernel RT manualmente

Una vez que `dpkg` termine sin errores, asegúrate de que el kernel que sí quieres usar tenga su imagen de arranque correcta:

```bash
update-initramfs -u -k 6.19.10+deb14-rt-amd64
```

¿Por qué sigue apareciendo `cloud` y `amd64`?

En tu salida de terminal dice: `Resumen: Actualizando: 0, Instalando 0... 7 no instalados del todo`. Esos "7 no instalados" son los que están causando el bucle. Al hacer el **paso 1** (`–purge`), esos paquetes desaparecen de la lista de tareas pendientes de `apt`.

¿Qué pasa si corres el comando del paso 2 y luego el 3? Eso debería desbloquearte el sistema inmediatamente. Una vez hecho eso, el `update-grub` solo mostrará los kernels que sobrevivieron a la purga.

2026-03-29 05:49:04 **You Asked**

[root@manjaro 2026-03-29 05:48:38 domingo ~]$
> dpkg –configure -a
Configurando initramfs-tools (0.150) ...
update-initramfs: deferring update (trigger activated)
Procesando disparadores para initramfs-tools (0.150) ...
update-initramfs: Generating /boot/initrd.img-6.19.8+deb14-rt-amd64
/usr/bin/update-initramfs: 149: dracut: not found
update-initramfs: failed for /boot/initrd.img-6.19.8+deb14-rt-amd64

with 127.
dpkg: error al procesar el paquete initramfs-tools (–configure):
 old initramfs-tools package postinst maintainer script subprocess failed with exit status 127
Se encontraron errores al procesar:
 initramfs-tools
[root@manjaro 2026-03-29 05:48:40 domingo ~]$
 > sudo mkdir -p /usr/lib/dracut/
sudo ln -s /usr/bin/true /usr/lib/dracut/dracut-install
[root@manjaro 2026-03-29 05:48:53 domingo ~]$
 > dpkg –configure -a
Configurando initramfs-tools (0.150) ...
update-initramfs: deferring update (trigger activated)
Procesando disparadores para initramfs-tools (0.150) ...
update-initramfs: Generating /boot/initrd.img-6.19.8+deb14-rt-amd64
/usr/bin/update-initramfs: 149: dracut: not found
update-initramfs: failed for /boot/initrd.img-6.19.8+deb14-rt-amd64 with 127.
dpkg: error al procesar el paquete initramfs-tools (–configure):
 old initramfs-tools package postinst maintainer script subprocess failed with exit status 127
Se encontraron errores al procesar:
 initramfs-tools

Gemini Fast

El problema es que el script `update-initramfs` está intentando ejecutar directamente el comando `dracut` (no solo el binario que creamos antes en `/usr/lib`), y al no encontrarlo en el `$PATH`, lanza el error **127**.

En tu sistema híbrido, `initramfs-tools` tiene un "secuestro" (hook) que redirige la generación hacia dracut. Vamos a usar la misma técnica del "engaño" pero para el comando global.

1. Crear un ejecutable falso para `dracut`

Esto engañará al script de Debian haciéndole creer que dracut corrió con éxito.

```bash
# Crear el enlace en /usr/bin para que el comando 'dracut' exista
sudo ln -s /usr/bin/true /usr/bin/dracut
```

```bash
# Asegurémonos de que el anterior también esté (por las dudas)
sudo mkdir -p /usr/lib/dracut/
sudo ln -sf /usr/bin/true /usr/lib/dracut/dracut-install
```

2. Intentar configurar de nuevo

Ahora que `dracut` devolverá siempre "éxito" (código 0), `dpkg` debería terminar su trabajo:

```bash
sudo dpkg --configure -a
```

3. Generar el Initramfs REAL (El paso más importante)

Si el paso anterior funciona, **CUIDADO**: el archivo `/boot/initrd.img-6.19.8...` no se habrá creado realmente (porque lo engañamos con `true`). Tu sistema **no arrancará** si reinicias ahora.

Debes forzar la creación del ramdisk usando la herramienta pura de Debian, ignorando los scripts automáticos:

```bash
# Generar la imagen real para el kernel RT 6.19.8
sudo mkinitramfs -o /boot/initrd.img-6.19.8+deb14-rt-amd64 6.19.8+deb14-rt-amd64

# Generar la imagen real para el kernel RT 6.19.10 (si es el que quieres usar ahora)
sudo mkinitramfs -o /boot/initrd.img-6.19.10+deb14-rt-amd64 6.19.10+deb14-rt-amd64
```

4. Actualizar GRUB

Una vez que los archivos `.img` estén creados en `/boot`, actualiza el menú:

```bash
sudo update-grub
```

¿Por qué sigue fallando?

El archivo `/usr/bin/update-initramfs` en la línea 149 tiene una instrucción que dice literalmente `dracut [opciones]`. Como el binario `dracut` no existe en tu sistema (porque prefieres no instalarlo), el script muere. Al hacer el link a `/usr/bin/true`, el script sigue de largo sin protestar.

¿Ves algún error al correr el `mkinitramfs` manual del paso 3? Si ese comando termina bien, ya estarás fuera de peligro para reiniciar.

2026-03-29 05:50:19 **You Asked**

> apt autoremove
ELIMINANDO:
libplymouth5

Resumen:
Actualizando: 0, Instalando 0, Eliminando: 1, no actualizando: 0
Espacio liberado: 497 kB

¿Continuar? [S/n]
(Leyendo la base de datos ... 213745 ficheros o directorios instalados actualmente.)
Desinstalando libplymouth5:amd64 (24.004.60-5.1) ...
Procesando disparadores para libc-bin (2.42-14) ...
Error: Se alcanzó el tiempo de expiración

Gemini Fast

El error de "Tiempo de expiración" (timeout) después de procesar los disparadores de `libc-bin` sugiere que el sistema está tardando demasiado en actualizar la caché de librerías dinámicas o que hay un problema de red persistente con los espejos de Debian Sid.

Sin embargo, lo más importante es que **ya desinstalaste `libplymouth5`**, lo que limpia un poco las dependencias visuales que a veces causan conflictos con los generadores de initramfs.

Para salir del estado de error y asegurar que tu "Frankeinux" arranque con el kernel que quieres, hagamos el "tiro de gracia" para arreglar el `initrd`:

1. El engaño final a Dracut

Como vimos que el script `update-initramfs` busca el comando `dracut` en la línea 149, vamos a crear el alias global para que deje de fallar:

```bash
sudo ln -sf /usr/bin/true /usr/bin/dracut
```

2. Reparar la configuración de paquetes

Ahora que `dracut` "existe" (como un comando que no hace nada), corre esto para que los paquetes de kernel se marquen como instalados correctamente:

```bash
sudo dpkg --configure -a
```

3. Generar el archivo de arranque REAL (Manual)

No confíes en el proceso automático porque lo engañamos con el paso anterior. Debes crear el archivo manualmente para que Btrfs sea reconocido:

```bash
# Para el kernel RT 6.19.10 (el más nuevo que instalaste)
sudo mkinitramfs -o /boot/initrd.img-6.19.10+deb14-rt-amd64 6.19.10+deb14-rt-amd64
```

Si este comando te da algún error de "archivo no encontrado", por favor pégamelo aquí.

4. Configurar el GRUB y Actualizar

Si quieres que el sistema arranque **siempre** con ese kernel RT específico (y evitar que intente cargar uno que falle), edita el archivo:

1. `sudo nano /etc/default/grub`
2. Asegúrate de que la línea sea: `GRUB_DEFAULT=0` (o el nombre que prefieras).
3. Asegúrate de que la línea de comandos esté limpia: `GRUB_CMDLINE_LINUX_DEFAULT="quiet splash audit=0"`
4. **Actualiza el menú:** `sudo update-grub`

¿Por qué el timeout en libc-bin? Al estar en **Debian Sid**, las actualizaciones de `glibc` son constantes y pesadas. Si el sistema se queda "pensando" mucho tiempo ahí, es probable que el disco esté saturado de operaciones de escritura o que el procesador Phenom II esté al 100% procesando los disparadores.

¿Lograste correr el `mkinitramfs` del paso 3 sin que te tire error 127? Si ese comando termina bien, ya puedes reiniciar tranquilo.

2026-03-29 05:51:40 **You Asked**

Quiero que la primera entrada del grub sea Debian GNU/Linux, with Linux 6.19-x86_64

Gemini Fast

Para lograr que esa entrada específica sea la primera (la opción por defecto `0`), debemos editar la configuración de las plantillas de GRUB. En Debian, el orden se determina alfabéticamente por la versión del kernel o por cómo se generan los archivos en `/etc/grub.d/`.

Como tienes un sistema híbrido, lo más seguro es definirlo por **nombre exacto** en lugar de por número, para que no cambie si instalas otro kernel por accidente.

1. Obtener el nombre exacto de la entrada

Primero, necesitamos saber cómo llama GRUB exactamente a ese kernel. Ejecuta este comando:

```bash
grep "menuentry 'Debian GNU/Linux, with Linux 6.19-x86_64'" /boot/grub/grub.cfg
```

Si no devuelve nada, prueba con `grep "menuentry" /boot/grub/grub.cfg` para ver la lista completa y copiar el nombre exacto.

---

2. Configurar el Kernel por defecto

Edita el archivo de configuración principal:

```bash
sudo nano /etc/default/grub
```

Modifica la línea `GRUB_DEFAULT` para que use el nombre que encontraste. Normalmente, si está dentro del submenú de "Advanced options", la sintaxis es:

```bash
GRUB_DEFAULT="Advanced options for Debian GNU/Linux>Debian GNU/Linux, with Linux 6.19-x86_64"
```

(Asegúrate de que el nombre coincida letra por letra con lo que devolvió el grep).

---

3. El paso de "supervivencia" (Importante)

Como venimos arrastrando el error de **dracut**, antes de que `update-grub` intente disparar los triggers de nuevo y falle, asegúrate de que el "engaño" siga activo:

```bash
```

```
# Si no lo hiciste antes, hazlo ahora:
sudo ln -sf /usr/bin/true /usr/bin/dracut
sudo ln -sf /usr/bin/true /usr/lib/dracut/dracut-install
```

---

4. Aplicar y Generar

Ahora sí, corre los comandos para reconstruir el menú con tu nueva prioridad:

```bash
# 1. Regenerar el initrd manualmente para ese kernel (por seguridad)
sudo mkinitramfs -o /boot/initrd.img-6.19-x86_64 6.19-x86_64

# 2. Actualizar el GRUB
sudo update-grub
```

Verificación final

Si el `update-grub` termina sin el error 127, al reiniciar verás que la opción resaltada automáticamente será la **6.19-x86_64**.

¿Te devolvió algo el comando `grep` del paso 1? Si me pasas el nombre exacto que aparece ahí, puedo darte la línea de `GRUB_DEFAULT` lista para copiar y pegar.

2026-03-29 05:52:27 **You Asked**

> grep "menuentry 'Debian GNU/Linux, with Linux 6.19-x86_64'" /boot/grub/grub.cfg
menuentry 'Debian GNU/Linux, with Linux 6.19-x86_64' --class debian --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-6.19-x86_64-advanced-cfb49c22-87f2-47d9-a25b-310d8d8578af' {

Gemini Fast

Perfecto, ya tenemos el identificador exacto. Para que esa entrada sea la primera y la predeterminada, vamos a configurar el GRUB usando el **ID único** que nos devolvió el grep, ya que es la forma más infalible de que no cambie.

Sigue estos pasos para dejarlo listo:

1. Editar la configuración de GRUB

Abre el archivo de configuración: `sudo nano /etc/default/grub`

Busca la línea `GRUB_DEFAULT` y reemplázala por lo siguiente (incluyendo las comillas):

```bash
GRUB_DEFAULT="gnulinux-advanced-cfb49c22-87f2-47d9-a25b-310d8d8578af>gnulinux-6.19-x86_64-advanced-cfb49c22-87f2-47d9-a25b-310d8d8578af"
```

> **Nota:** Esta sintaxis le dice al GRUB: "Entra al submenú de opciones avanzadas (`>`) y selecciona el kernel 6.19 con ese ID específico".

2. Asegurar el arranque (Btrfs e Initrd)

Como vimos que los procesos automáticos fallan por el tema de Dracut, antes de actualizar el GRUB, asegúrate de que el archivo de imagen de ese kernel exista y sea válido:

```bash
# Generar el initrd manual para el kernel 6.19-x86_64
sudo mkinitramfs -o /boot/initrd.img-6.19-x86_64 6.19-x86_64
```

3. Aplicar los cambios

Ahora aplica la configuración para que el menú se reconstruya con tu preferencia:

```bash
sudo update-grub
```

Resumen de cómo queda tu archivo corregido

Si quieres el archivo completo y limpio para pegar en `/etc/default/grub`, debería verse así:

```bash
# Archivo corregido para Frankeinux - Kernel 6.19 Default
GRUB_DEFAULT="gnulinux-advanced-cfb49c22-87f2-47d9-a25b-310d8d8578af>gnulinux-6.19-x86_64-advanced-cfb49c22-87f2-47d9-a25b-310d8d8578af"
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash audit=0"
GRUB_CMDLINE_LINUX=""
```

```
# Configuración de terminal y video
GRUB_TERMINAL_INPUT="console serial"
GRUB_TERMINAL_OUTPUT="gfxterm serial"
GRUB_SERIAL_COMMAND="serial --speed=115200"
GRUB_GFXMODE=1280x1024x32,1024x768,auto
GRUB_GFXPAYLOAD_LINUX=keep
```

Verificación final antes de reiniciar

Corre este comando para confirmar que el archivo de imagen se creó bien y tiene soporte para Btrfs: `lsinitramfs /boot/initrd.img-6.19-x86_64 | grep btrfs`

Si te devuelve líneas con "btrfs", el sistema arrancará sin el error de la foto que mandaste antes.

¿Te gustaría que verifiquemos si el `update-grub` terminó sin errores esta vez?

2026-03-29 05:53:57 **You Asked**

No quiero que seleccione, quiero que aparezca como primer entrada en el grub.

Gemini Fast

Entiendo perfectamente. No quieres que el GRUB "salte" al submenú para elegirlo, sino que el kernel **6.19-x86_64** sea la **primerísima opción** que aparece apenas prendés la pantalla, sin entrar en "Advanced options".

En Debian/Frankeinux, el script `/etc/grub.d/10_linux` ordena los kernels por versión (de mayor a menor). Si el 6.19 no está quedando primero es porque GRUB detecta otra versión como "más nueva" o porque el script está priorizando otras imágenes.

Para forzarlo a ser el primero de la lista, vamos a usar el método del **Script Personalizado**:

1. Crear una entrada prioritaria

Vamos a crear un script que ponga el kernel 6.19 antes que todos los demás.

1. Crea un nuevo archivo de configuración con prioridad alta (número bajo, como `09`): `sudo nano /etc/grub.d/09_custom_kernel`
2. Pega el siguiente contenido (basado en el `menuentry` que encontraste con el grep):

```bash
#!/bin/sh
exec tail -n +3 $0
# Esta es la entrada manual para el kernel 6.19-x86_64
menuentry 'Debian GNU/Linux (Kernel 6.19-x86_64)' --class debian --class gnu-linux --class gnu
--class os {
    load_video
    insmod gzio
    insmod part_msdos
    insmod btrfs
    set root='hd0,msdos1'
    search --no-floppy --fs-uuid --set=root cfb49c22-87f2-47d9-a25b-310d8d8578af
    echo    'Cargando Linux 6.19-x86_64 ...'
    linux   /@/boot/vmlinuz-6.19-x86_64 root=UUID=cfb49c22-87f2-47d9-a25b-310d8d8578af ro
rootflags=subvol=@ quiet splash audit=0
    echo    'Cargando ramdisk inicial ...'
    initrd  /@/boot/initrd.img-6.19-x86_64
}
```

3. Dale permisos de ejecución para que GRUB lo tome en cuenta:
   `sudo chmod +x /etc/grub.d/09_custom_kernel`

2. Ajustar el `/etc/default/grub`

Ahora que creamos una entrada manual que aparecerá antes que las automáticas, vuelve a poner el default en 0:

1. `sudo nano /etc/default/grub`
2. Cambia la línea a: **`GRUB_DEFAULT=0`**
3. (Opcional) Si quieres ocultar el submenú "Advanced options" para que no moleste, puedes añadir al final: `GRUB_DISABLE_SUBMENU=y`

3. El paso de seguridad (Reparar el Initrd)

Como tu sistema tiene el conflicto con **dracut**, asegurate de que el archivo que declaramos en el script (`initrd.img-6.19-x86_64`) realmente exista y tenga los módulos de Btrfs, de lo contrario volverás a la pantalla de error.

```bash
# Engaño a dracut (si no lo hiciste antes)
sudo ln -sf /usr/bin/true /usr/bin/dracut

# Generación manual del initrd para ese kernel
sudo mkinitramfs -o /boot/initrd.img-6.19-x86_64 6.19-x86_64
```

4. Actualizar y Verificar

Finalmente, corre el comando para que GRUB lea tu nuevo script `09_custom_kernel`:

```bash
sudo update-grub
```

¿Qué logramos con esto?

Al usar el número **09**, GRUB ejecutará tu script antes que el `10_linux` (que es el que genera la lista automática). Esto garantiza que el kernel 6.19 sea la **entrada #0** de la lista principal.

¿Viste algún error al correr el `update-grub`? Si terminó bien, al reiniciar deberías ver "Debian GNU/Linux (Kernel 6.19-x86_64)" como la primera opción de todas.

2026-03-29 05:54:55 **You Asked**

Lo tengo así.

```
> ls -lh /etc/grub.d/
total 200K
-rwxr-xr-x 1 root root  11K feb 10 08:27 00_header
-rwxr-xr-x 1 root root 9,6K ene 15 06:03 00_header.dpkg-old
-rwxr-xr-x 1 root root 6,2K feb 10 08:27 05_debian_theme
-rwxr-xr-x 1 root root  14K feb 10 08:27 10_linux
-rwxr-xr-x 1 root root  13K ene 15 06:03 10_linux.dpkg-old
lrwxrwxrwx 1 root root   34 mar 21 01:56 15_ostree ->
/usr/lib/libostree/grub2-15_ostree
-rwxr-xr-x 1 root root  15K feb 10 08:27 20_linux_xen
-rwxr-xr-x 1 root root  15K ene 15 06:03 20_linux_xen.dpkg-old
-rwxr-xr-x 1 root root  786 ene 15 06:03 25_bli
-rwxr-xr-x 1 root root  13K feb 10 08:27 30_os-prober
-rwxr-xr-x 1 root root  13K mar 27 16:27 30_os-prober.dpkg-old
-rwxr-xr-x 1 root root 1,2K feb 10 08:27 30_uefi-firmware
-rwxr-xr-x 1 root root 1,1K mar 21 01:56 30_uefi-firmware.dpkg-old
-rwxr-xr-x 1 root root  744 feb 26 09:49 35_fwupd
-rwxr-xr-x 1 root root  730 mar 21 01:56 35_fwupd.dpkg-old
-rwxr-xr-x 1 root root  214 ene 15 06:03 40_custom
-rwxr-xr-x 1 root root  215 ene 15 06:03 41_custom
```

```
-rwxr-xr-x 1 root root  28K mar 21 01:56 41_snapshots-btrfs
-rwxr-xr-x 1 root root 1,2K mar 21 01:56 60_memtest86+
-rwxr-xr-x 1 root root 1,4K mar 21 01:56 60_memtest86+-efi
-rw-r--r-- 1 root root  483 ene 15 06:03 README
```

Gemini Fast

Para que el kernel **6.19-x86_64** aparezca como la primera opción absoluta en el menú de GRUB, vamos a aprovechar que el orden de los archivos en `/etc/grub.d/` es numérico. Actualmente, el script `10_linux` es el que genera la lista automática.

Si creamos un script con un número menor (por ejemplo, `08`), GRUB lo pondrá al principio de todo.

Paso 1: Crear el script de entrada prioritaria

Copia y pega este comando para crear el archivo directamente con el contenido necesario:

```bash
sudo tee /etc/grub.d/08_kernel_fijo << 'EOF'
#!/bin/sh
exec tail -n +3 $0
# Entrada manual para que sea la primera en la lista
menuentry 'Debian GNU/Linux (Kernel 6.19-x86_64)' --class debian --class gnu-linux --class gnu
--class os {
    load_video
    insmod gzio
    insmod part_msdos
    insmod btrfs
    set root='hd0,msdos1'
    search --no-floppy --fs-uuid --set=root cfb49c22-87f2-47d9-a25b-310d8d8578af
    echo    'Cargando Linux 6.19-x86_64 ...'
    linux   /@/boot/vmlinuz-6.19-x86_64 root=UUID=cfb49c22-87f2-47d9-a25b-310d8d8578af ro
rootflags=subvol=@ quiet splash audit=0
    echo    'Cargando ramdisk inicial ...'
    initrd  /@/boot/initrd.img-6.19-x86_64
}
EOF
```

Paso 2: Darle permisos de ejecución

Sin esto, GRUB lo ignorará:

```bash
sudo chmod +x /etc/grub.d/08_kernel_fijo
```

Paso 3: Limpiar el archivo `/etc/default/grub`

Para que no haya confusiones con el ID largo que buscamos antes, deja el archivo simple:

1. `sudo nano /etc/default/grub`
2. Asegúrate de que diga: `GRUB_DEFAULT=0`
3. Guarda y sal.

Paso 4: Reparar el entorno y actualizar

Antes de correr el `update-grub`, recordá el bloqueo de **dracut** que tenés. Si no lo engañamos, el comando va a fallar de nuevo:

```bash
# El "engaño" necesario para tu sistema híbrido
sudo ln -sf /usr/bin/true /usr/bin/dracut
sudo ln -sf /usr/bin/true /usr/lib/dracut/dracut-install

# Generar el initrd manual por las dudas
sudo mkinitramfs -o /boot/initrd.img-6.19-x86_64 6.19-x86_64

# Aplicar cambios al GRUB
sudo update-grub
```

¿Cómo va a quedar tu lista de archivos ahora?

Cuando hagas `ls -lh /etc/grub.d/`, verás tu nuevo archivo `08_kernel_fijo` antes del `10_linux`.

Esto garantiza que al reiniciar, la **primera línea del menú** sea "Debian GNU/Linux (Kernel 6.19-x86_64)". El resto de los kernels seguirán apareciendo abajo (generados por el script 10), pero el tuyo siempre ganará la posición #0.

¿Querés que verifiquemos si el archivo `/boot/vmlinuz-6.19-x86_64` existe con ese nombre exacto antes de reiniciar? Solo por seguridad.